Gesällprov: Connect 5

Alexander Herder, alhe5785

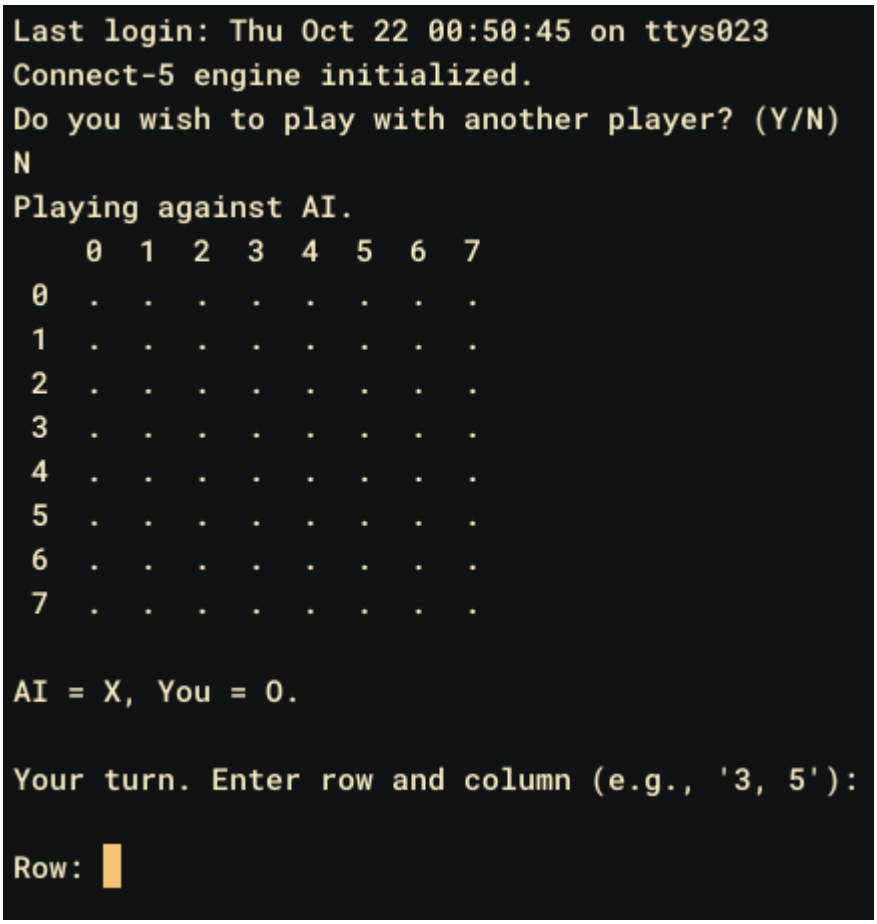
Sammanfattning

Detta är ett enkelt Connect 5-spel (5-i-rad-spel) med både singleplayer- och multiplayerfunktionalitet. I singleplayer-läget spelas spelet mot en AI som använder sig av minimaxalgoritmen för att kunna välja bästa draget. I multiplayer-läget kan en spelare starta en “game session” (spelsession). Varje session kan ha max två spelare, och ett Connect 5-spel kan bara startas om två spelare är anslutna till samma session. Som spelare kan man också ansluta till någon annans redan startade session genom att skriva in en room code (rumskod) som sessionsstartaren får utskriven som terminaloutput när en session startas. För att en session ska kunna startas behövs en serverklass köras först, vilket finns i detta projekt också.

```
Last login: Thu Oct 22 00:50:39 on ttys016
Connect-5 engine initialized.
Do you wish to play with another player? (Y/N)
Y
Type CREATE if you wish to create a new room.
Type JOIN if you wish to join another player's r
oom.
JOIN
Please enter room code of game you wish to join.
jfiuvi
Player 2 joined.
    0  1  2  3  4  5  6  7
 0  .  .  .  .  .  .  .  .
 1  .  .  .  .  .  .  .  .
 2  .  .  .  .  .  .  .  .
 3  .  .  .  .  .  .  .  .
 4  .  .  .  .  .  .  .  .
 5  .  .  .  .  .  .  .  .
 6  .  .  .  .  .  .  .  .
 7  .  .  .  .  .  .  .  .

Player 1 = X, Player 2 = O
Waiting for other player to make their move...
```

Hur konsolloutputen ser ut för spelare 2 när en multiplayer-match startar.

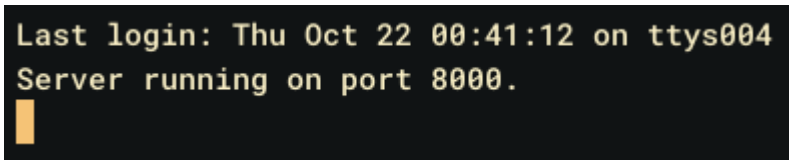


Hur konsolloutputen ser ut när en spelare och en AI möter varandra.

Funktion

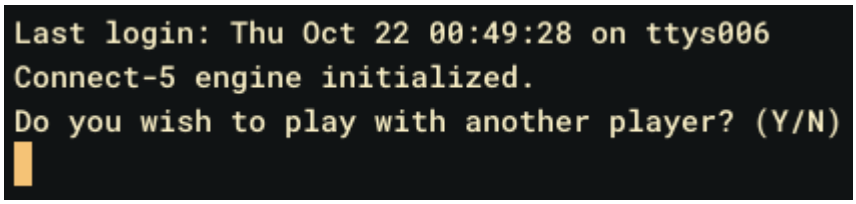
Start

Programmet körs helt i terminalen. För multiplayer måste servern startas först (`server.Server`), som sedan lyssnar på port 8000 och skriver ut `Server running on port 8000.`:



Servern har startats och väntar på klienter.

Efter det startar varje spelare en egen klient (`client.Game`). Klienten frågar direkt om man vill spela mot en annan spelare eller inte:



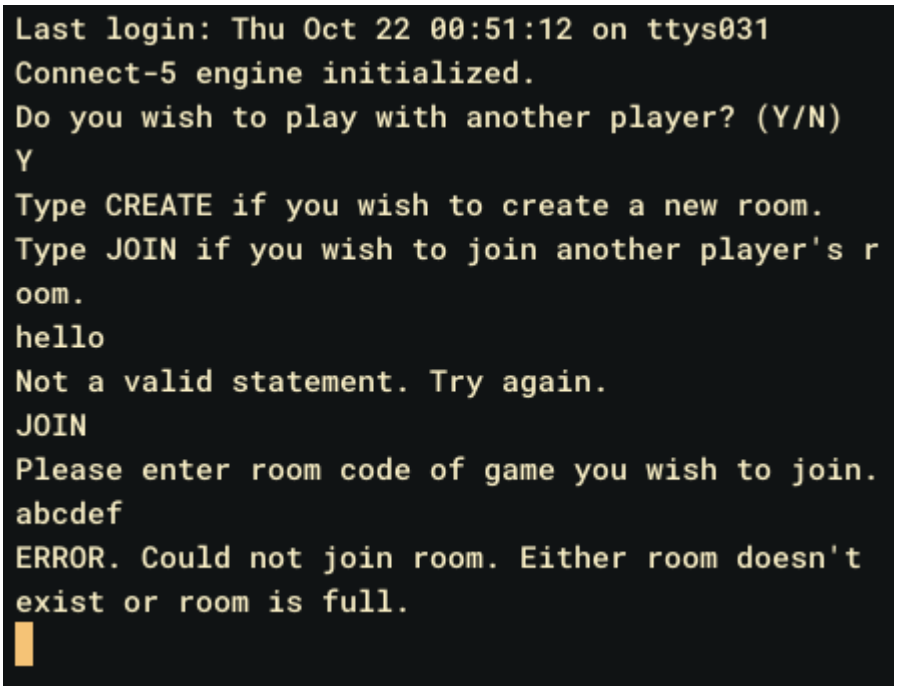
Startfrågan som visas när klienten startas.

Svarar man `Y` startas multiplayer-läget och klienten ansluter till servern. Svarar man `N` startas singleplayer-läget mot AI:n, och då används ingen server alls.

Skapa och gå med i en match

I multiplayer-läget väljer spelaren mellan `CREATE` och `JOIN`. Skriver man något annat får man ett felmeddelande och kan försöka igen. Den som skriver `CREATE` blir spelare 1 och får en rumskod på sex bokstäver utskriven. Den koden skickar man till den andra spelaren, som skriver `JOIN` och sedan koden. När spelare 2 har anslutit får båda spelarna meddelandet `Player 2 joined.` och matchen startar direkt. Bilden under Sammanfattning visar hur det ser ut för spelare 2 efter att ha gått med i ett rum: bekräftelsen, det tomma brädet och väntemeddelandet medan spelare 1 gör sitt första drag.

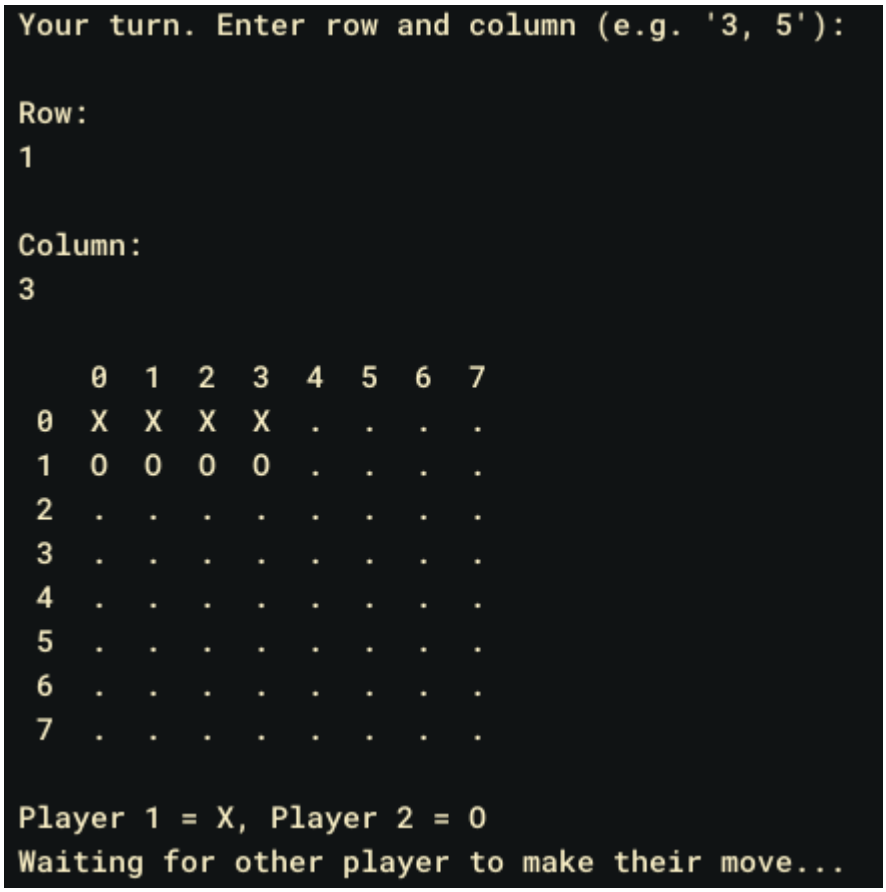
Om rumskoden inte finns, eller om rummet redan har två spelare, svarar servern med ett felmeddelande och klienten avslutas:



Ett ogiltigt kommando följt av ett JOIN till ett rum som inte finns.

Ett drag

Brädet är 8×8 och skrivs ut med rad- och kolumnnummer. Spelare 1 är `X` och spelare 2 är `0`, och spelare 1 börjar alltid. Den spelare som har turen får skriva in rad och kolumn var för sig, medan den andra ser `Waiting for other player to make their move...`. Draget kontrolleras först lokalt hos klienten: ligger det utanför brädet eller på en redan upptagen ruta får spelaren ett felmeddelande och får försöka igen, utan att något skickas till servern.

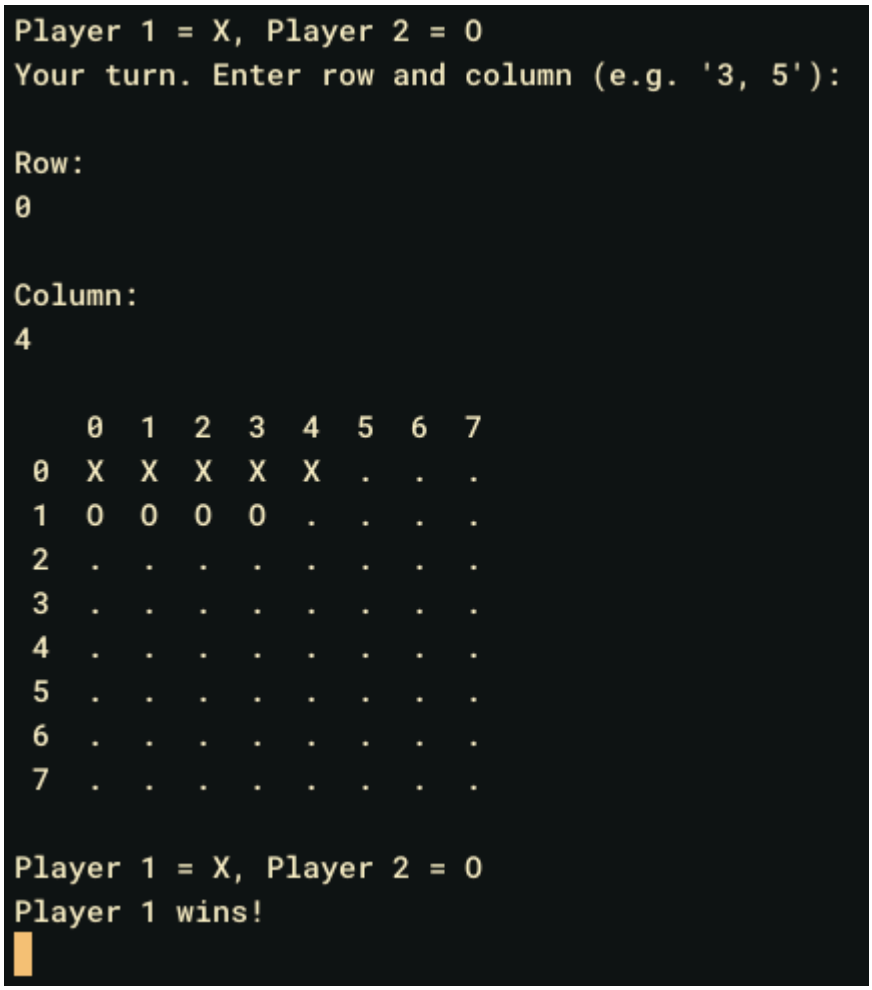


Spelare 2 har just lagt sin pjäs på rad 1, kolumn 3. Brädet har uppdaterats och spelare 2 väntar nu på spelare 1:s drag.

När ett giltigt drag har skickats uppdaterar servern sitt bräde och skickar det nya brädet till båda spelarna. Därför ser båda samma bräde efter varje drag, och turen går över till den andra spelaren.

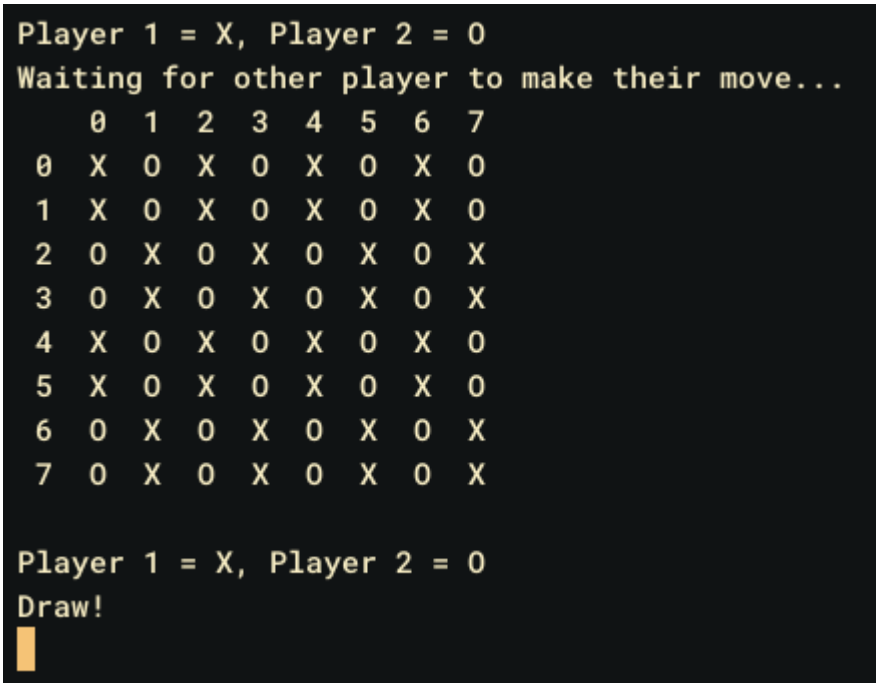
Vinst och oavgjort

Efter varje drag kontrollerar servern om någon har fem i rad, vågrätt, lodrätt eller diagonalt. I så fall skickas ett vinstmeddelande till båda spelarna och klienterna avslutas:



Spelare 1 vinner med fem i rad på rad 0. Samma meddelande visas hos båda spelarna.

Om hela brädet fylls utan att någon fått fem i rad blir det oavgjort, och båda spelarna får `Draw!`:



En match där alla 64 rutor fyllts utan vinnare.

Singleplayer

I singleplayer-läget spelar man `O` mot AI:n, som är `X`. Människan börjar. Inmatningen fungerar på samma sätt som i multiplayer, med samma kontroller för ogiltiga drag. När det är AI:ns tur skrivs `AI is thinking...` ut medan minimax-algoritmen söker fem drag framåt. Det tar oftast bara någon sekund per drag. Tidigare var sökdjupet sex drag, men då tog varje drag ungefär en minut eftersom sökträdet växer så snabbt, så djupet sänktes till fem. När spelet är slut skrivs `You win!`, `You lose!` eller `Draw!` ut. Hur det ser ut syns i den andra bilden under Sammanfattning.

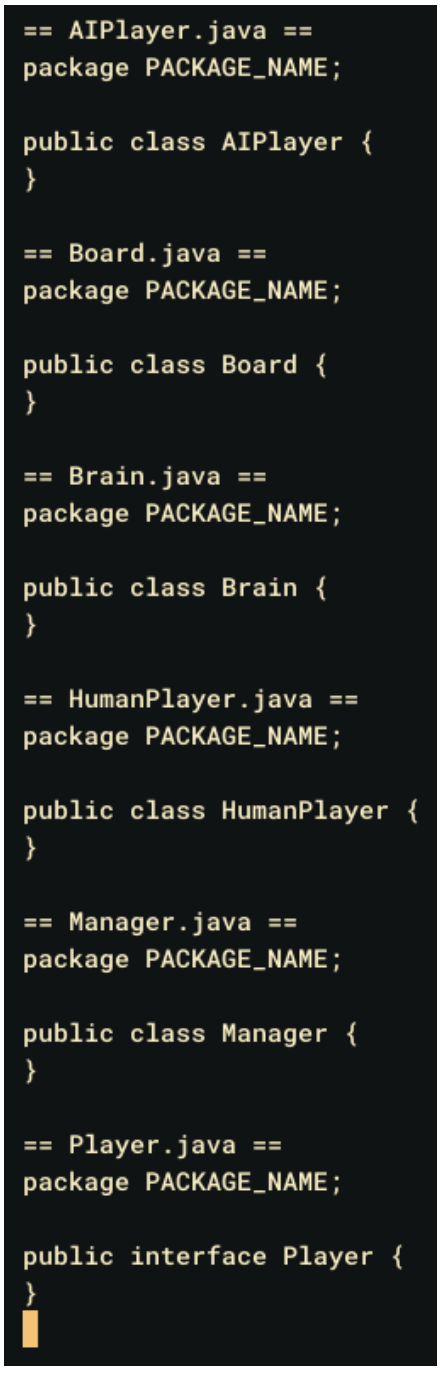
Kända begränsningar

- Om en spelare kopplar ner mitt i en match avbryts sessionen på servern, men den andra spelaren får inget meddelande och blir kvar och väntar på ett drag som aldrig kommer.

Framtagande

Grundversionen av spelet, med brädet, vinstkontrollen och AI:n med minimax, skrevs i mars 2026 som en uppgift i kursen ALDA. Som gesällprov i IPROG har jag byggt vidare på den med det som kursen handlar om: en server med spelsessioner och rumskoder, kommunikation över sockets mellan klient och server, trådar för varje match och ett eget protokoll för spelet.

Initial idé



```
== AIPlayer.java ==
package PACKAGE_NAME;

public class AIPlayer {
}

== Board.java ==
package PACKAGE_NAME;

public class Board {
}

== Brain.java ==
package PACKAGE_NAME;

public class Brain {
}

== HumanPlayer.java ==
package PACKAGE_NAME;

public class HumanPlayer {
}

== Manager.java ==
package PACKAGE_NAME;

public class Manager {
}

== Player.java ==
package PACKAGE_NAME;

public interface Player {
}
```

Den första skissen av arkitekturen (första commiten, mars 2026): tomma klasser för `Board`, `Brain`, `Manager`, `AIPlayer`, `HumanPlayer` och ett `Player`-interface.

Min första tanke var en klassisk objektorienterad uppdelning, där människan och AI:n var två olika implementationer av ett `Player`-interface och en `Manager` styrde spelet. Jag tog bort den uppdelningen ganska snabbt. Det fanns ingen poäng med att skilja på spelare och AI, eftersom spelloopen ändå bara behöver veta vems tur det är, och en klass som heter `Manager` hade varit dålig namngivning. Istället blev det en `Game`-klass som styr spelet, med `Board` och `Brain` kvar från skissen.

```java
public class Board {
    /*
    representing board pieces with two bits (00 = empty, 01 = player 1, 10 = player 2).
    we want a 15x15 grid, which is 225 squares. each square is 2 bits, so we need 450 bits.
    a long is 64 bits, and 64 x 8 = 512 which 450 fits into.
     */
    private final long[] board = new long[8];
    private final int BOARD_HEIGHT = 15;
    private final int PIECE_LENGTH = 2;

    /*
    choosing to set pieces using bit manipulation to optimize performance
    ( and also because I thought it was really fun in CPROG :) )
     */
    public void setPiece(int row, int col, int player) {
        if (player < 0 || player > 2) {
            throw new IllegalArgumentException("Player must be either 1, 2 or 0 to clear piece.");
        }

        int arrayIndex = getIndex(row, col);
        // where exactly is the leading bit of the piece?
        int bitOffset = (row * (BOARD_HEIGHT * PIECE_LENGTH) + col * PIECE_LENGTH) % 64;

        // make a mask clearing bits we want to overwrite
        long clearMask = ~(3L << bitOffset);

        // clear bits for board
        board[arrayIndex] &= clearMask;

        // prepare mask for which player to apply to piece
        long pieceMask = ((long)player) << bitOffset;

        // place player mask into correct piece
        board[arrayIndex] |= pieceMask;
    }

    public int getIndex(int row, int col) {
        // divide by 64 to get the absolute first bit of the long, since a long is 64 bits
        return (row * (BOARD_HEIGHT * PIECE_LENGTH) + col * PIECE_LENGTH) / 64;
    }
}
```

`Board` och `setPiece()` i en tidig version. Brädet var då hårdkodat till 15×15 och kommentarerna visar uträkningen av hur många `long`-värden som behövs.

I början tänkte jag använda mig av en vanlig 2d-intarray för att representera spelplanen/brädan (Board), men eftersom minimax-algoritmen söker så oerhört många olika game states så valde jag att använda mig av en enda long[] för att göra varje lookup så snabb och enkel som möjligt. Det gjorde de egna metoderna för att hämta bitar och sätta bitar lite svårare att skriva, men det var värt för optimeringens skull. Det var också väldigt roligt att finslipa mina kunskaper inom bitmanipulering.

Implementation av brädlogik

```java
/*
helper method to check 5 in a row for a certain direction.
dRow is direction of row, dCol is direction of column
0, 1 to move right (horizontal)
1, 0 to move down (vertical)
1, 1 to move down-right
-1, 1 to move up-right

returns true if there are n in a row for player, where n = length
 */

public boolean checkDirection(int startRow, int startCol, int dRow, int dCol, long player, int length) {

    // -1 because starting piece counts as first piece
    int endRow = startRow + ((length - 1) * dRow);
    int endCol = startCol + ((length - 1) * dCol);

    if (endRow > board.getSize() - 1 || endRow < 0 || endCol < 0 || endCol > board.getSize() - 1) {
        return false;
    }

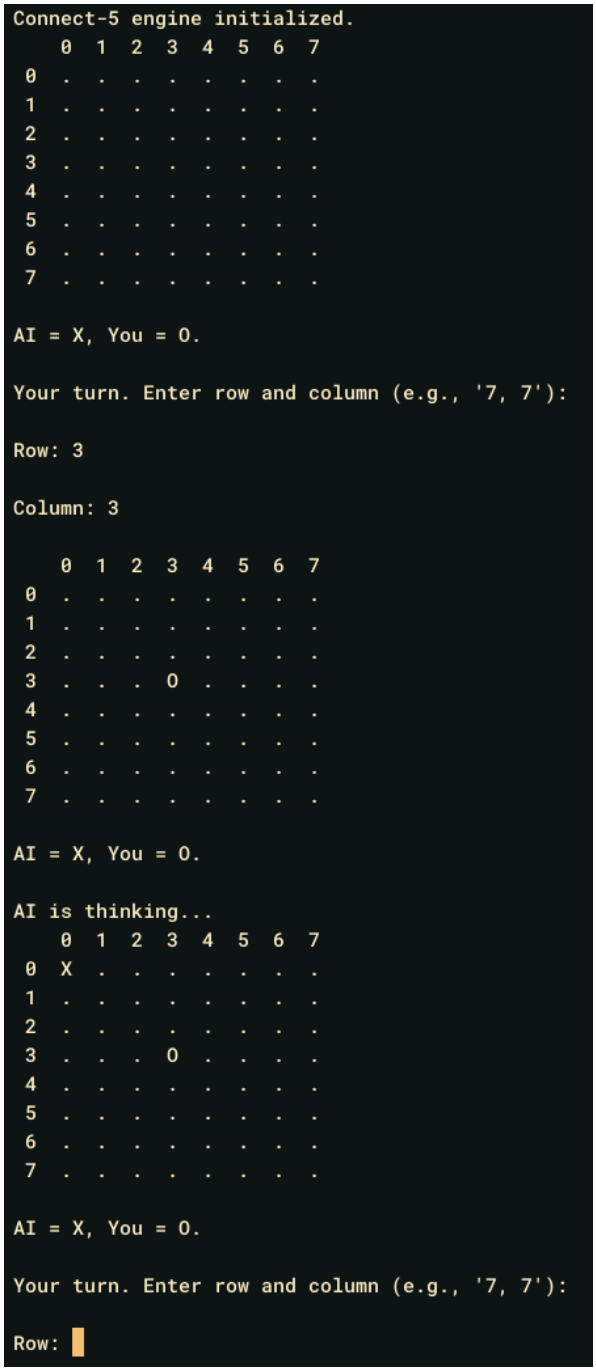
    int count = 0;
    int row = startRow;
    int col = startCol;

    while (count < length) {
        if (board.getPiece(row, col) == player) {
            count++;
        } else {
            return false;
        }
        row += dRow;
        col += dCol;
    }
    return count == length;
}
```

`checkDirection()` med lathunden för `dRow` och `dCol` som kommentar ovanför metoden.

Efter den initiala bräd-representationen implementerade jag `checkDirection()` och `checkWinner()` i `Brain`, samt gjorde `Board` mer generell: storlek och pjäslängd (`size`, `pieceLength`) blev konstruktorparametrar istället för hårdkodade konstanter, och jag lade till `getLegalMoves()` samt delade upp indexeringen i en publik `getIndex()` och en privat `getInternalIndex()`. `checkDirection()` skrevs som en enda generell hjälpmetod som tar en riktning (`dRow`, `dCol`) istället för separata metoder för horisontellt/vertikalt/diagonalt, och `checkWinner()` anropar den i fyra riktningar (höger, ner, ner-höger, upp-höger) för varje ruta på brädet. Det var lite krångligt att komma på att jag skulle generalisera `checkDirection()`-metoden. I början tänkte jag att man skulle ha tre olika `checkDirection()`-metoder: en för att kolla horisontellt, en för vertikalt och en för diagonala kollar. Snabbt insåg jag att det hade varit mycket smidigare att bara specificera riktning i metodparametrarna. Det tog en liten stund att komma på hur jag skulle göra att det funkade, men det löste sig till slut. En lathund för hur man använder metoden ligger som kommentar utanför metoden, mest för att jag själv skulle komma ihåg hur jag tänkte med riktnings-parametrarna (`dRow`, `dCol`).

Implementation av minimax med alpha-beta-pruning



Den färdiga grundversionen från mars 2026 under körning. Här fanns bara singleplayer, så programmet går direkt in i ett spel mot AI:n utan att fråga om multiplayer.

Varför jag valde minimax var för att det just var det som lärdes ut under kursen jag gick (ALDA) när grundversionen av programmet skulle implementeras, och vi behövde använda den för att få godkänt på uppgiften. Sedan vet jag inte om det finns ett bättre sätt att göra detta på, kanske via någon sorts lookup men då hade data behövts insamlas för det vilket jag inte hade tid för.

Utökning med nätverksfunktionalitet

Det som saknades var att kunna spela med folk, så då valde jag att göra det som mitt Gesällprov. Mellan att den initiala implementationen blev klar (mars 2026) och att nätverksfunktionaliteten byggdes (september 2026) låg det ett uppehåll på ungefär fem månader. Detta eftersom jag hade sommaruppehåll under de månaderna, och inte än hade bestämt mig för att basera IPROG-gesällprovet på Connect5-spelet.

Uppdelning i client/server/shared



```
// commenting order of stream writes/reads to keep track of what comes when,
// that's what the commented numbers mean
@Override
public void run() {
    int playerTurn = 1;

    try {

        DataOutputStream outputStream1 = new DataOutputStream(player1Socket.getOutputStream());
        DataOutputStream outputStream2 = new DataOutputStream(player2Socket.getOutputStream());

        DataInputStream inputStream1 = new DataInputStream(player1Socket.getInputStream());
        DataInputStream inputStream2 = new DataInputStream(player2Socket.getInputStream());

        while (true) {

            long[] boardState = board.getBoard();

            for (int i = 0; i < boardState.length; i++) {
                outputStream1.writeLong(boardState[i]); // 1
                outputStream2.writeLong(boardState[i]); // 1
            }

            long winner = board.checkWinner();

            if (winner != 0) {
                outputStream1.writeUTF("Player " + winner + " wins!"); // 2
                outputStream2.writeUTF("Player " + winner + " wins!"); // 2
                break;
            } else {

                // sending player turn to both players so they can check
                // themselves if they're allowed to play
                outputStream1.writeInt(playerTurn); // 2
                outputStream2.writeInt(playerTurn); // 2

                // inner loop to not have to reinitialize boardState and turn messages
                while (true) {
                    int row = 0;
                    int col = 0;

                    DataInputStream activeInputStream;
                    DataOutputStream activeOutputStream;

                    if (playerTurn == 1) {
                        activeInputStream = inputStream1;
                        activeOutputStream = outputStream1;

                    } else {
                        activeInputStream = inputStream2;
                        activeOutputStream = outputStream2;
                    }

                    row = activeInputStream.readInt(); // 1
                    col = activeInputStream.readInt(); // 2

                    // bounds checks for placed pieces are locally handled

                    board.setPiece(row, col, playerTurn);
                    break;
                }

                // if 1 has turn, 3 - 1 = 2. if 2 has turn, 3 - 2 = 1. Elegant!
                playerTurn = 3 - playerTurn;

            }
        }

    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
```

Matchloopen i den första versionen av `GameSession` (september 2026). Siffrorna `// 1` och `// 2` i kommentarerna är numreringen av läsningar och skrivningar som beskrivs nedan.

Jag ville mest bara lära mig mer om hur det funkar med client/server-logik, och hur den implementeras i lite större skala. Det innebar att `Game.java` delades upp i en `client`-variant, att `Server` och `GameSession` skapades från grunden på serversidan, och att `Board`/`Brain` flyttades till ett gemensamt `shared`-paket som både klient och server använder.

Något som var oväntat svårt med implementeringen var att hålla koll på vilka read/writes som hörde till varandra. Jag behövde numrera varje anrop för att hitta ordningen; något som tog upp en större del av den totala arbetstiden än vad jag hade önskat. Annars gick detta ganska smidigt, och när jag väl hade delat upp allt i klasserna jag ville ha kom resten av koden ganska naturligt.

Testning och buggfixar

```
diff --git a/src/server/GameSession.java b/src/server/GameSession.java
index 6c9d9f8..d90fcca 100644
--- a/src/server/GameSession.java
+++ b/src/server/GameSession.java
@@ -68,6 +68,13 @@ public class GameSession implements Runnable {
                    outputStream1.writeUTF("Player " + winner + " wins!"); // 2
                    outputStream2.writeUTF("Player " + winner + " wins!"); // 2
                    break;
+
+                   // draw
+               } else if (board.hasNoLegalMovesLeft()) {
+                   outputStream1.writeUTF("Draw!");
+                   outputStream2.writeUTF("Draw!");
+                   break;
+
                } else {

                    // sending player turn to both players so they can check
@@ -75,31 +82,24 @@ public class GameSession implements Runnable {
                    outputStream1.writeInt(playerTurn); // 2
                    outputStream2.writeInt(playerTurn); // 2

-                   // inner loop to not have to reinitialize boardState and turn messages
-                   while (true) {
:...skipping...
diff --git a/src/server/GameSession.java b/src/server/GameSession.java
```

En del av ändringen i `GameSession` som la till hantering av oavgjort i multiplayer-läget (oktober 2026). Gröna rader är tillagda och röda är borttagna.

När jag skrev pseudokoden för beskrivningen gick jag igenom koden steg för steg, och då hittade jag flera brister. Multiplayer-läget hade ingen kontroll för oavgjort, så om brädet blev fullt utan vinnare fortsatte spelet att be om drag som inte gick att göra. En spelare som försökte gå med i ett rum som inte fanns, eller som redan var fullt, fick inget svar alls och blev hängande. Den inre loopen i `GameSession` visade sig också alltid köras exakt en gång, eftersom ogiltiga drag redan stoppas hos klienten, så den togs bort. Till sist upptäckte jag, när jag tog skärmdumparna till den här beskrivningen, att hela servern stängdes av om en klient kopplade ner innan den hade skickat `CREATE` eller `JOIN`. Felhanteringen omslöt hela serverloopen, så den flyttades till att gälla en klient i taget. Alla fyra fixarna testades genom att köra servern och klienterna och spela igenom just de fallen.

Form

Paketstruktur

Koden är uppdelad i tre paket: `client`, `server` och `shared`.

- `client` innehåller `Game`, som är spelarens ingångspunkt oavsett om man spelar singleplayer eller multiplayer. Den tar emot inmatning, skriver ut brädet och sköter i multiplayer-läget kommunikationen med servern.
- `server` innehåller `Server`, som tar emot anslutningar och hanterar `CREATE`- och `JOIN`-förfrågningar, samt `GameSession`, som kör en match mellan två spelare i en egen tråd.
- `shared` innehåller `Board` och `Brain`, som används av både klient och server. `Board` håller reda på brädet, vilka drag som är tillåtna och om någon har vunnit. `Brain` är AI:n, som väljer drag med minimax-algoritmen och alpha-beta-pruning. Eftersom klient och server använder samma `Board` finns spelreglerna bara på ett ställe.

Nätverksprotokoll

Flödet för en multiplayer-match:

1. Klienten ansluter till servern och skickar `CREATE` eller `JOIN`. Vid `JOIN` skickas även rumskoden.
2. Vid `CREATE` skapar servern en ny spelsession med en slumpad rumskod och skickar tillbaka koden. Klienten blir spelare 1.
3. Vid `JOIN` letar servern upp sessionen. Om den finns och inte är full blir klienten spelare 2, båda spelarna får en bekräftelse och sessionen startas i en egen tråd. Annars får klienten ett felmeddelande och avslutas.
4. I varje runda skickar sessionen brädet till båda spelarna.
5. Om någon har vunnit, eller om brädet är fullt, skickas resultatet till båda spelarna och matchen avslutas. Annars skickas vems tur det är.
6. Spelaren vars tur det är skickar sitt drag, servern lägger in det på brädet och nästa runda börjar.

Hur programmet fungerar i pseudokod

```
# Server
sessions start empty

loop

  pause loop until client arrives

  request = getClientOutput()
  if request == "CREATE" then
    roomCode = random six letters
    sessions(roomCode) = new GameSession with matching room code
    send room code to client

  else

    if request == "JOIN" then
      roomCode = getClientOutput()
      session = sessions(roomCode)

      if session exists and that session doesn't have a player 2 then
        assign to matching room
        broadcast "Player 2 joined." to both clients
        GameSession(session)&

      else

        send error message to client that tried to join unavailable room

return ""
```

```
# GameSession

playerTurn = 1

the board starts empty

loop
  send board state to clients

  if there is a winner then
    send winner message to both clients
    exit

  else
    if there is a draw then
      send draw message to both clients
      exit

    else
      send playerTurn to both clients
      read row and col variables from player whose turn it is
      set player piece on internal board

  flip playerTurn to other player

return ""
```

```
# Game

if player wants multiplayer game then
  startMultiplayerGame()

else
  if player wants singleplayer game then
    startSingleplayerGame()


return ""

# startMultiplayerGame
connect to Server

loop
  ask user if they want to CREATE or JOIN room

  answer = user input

  if answer == "CREATE" or answer == "JOIN" then
    exit
  else
    show error message

if answer == "CREATE" then
  send "CREATE" to Server
  get room code from Server
  show room code to client terminal

if answer == "JOIN" then
  send "JOIN" to Server
  ask user for room code
  send room code to Server

receive confirmation
show confirmation to user

if confirmation != "Player 2 joined." then
  return ""

loop
  receive board from GameSession
  print board to client terminal

  if there is winner or draw then
    show result message from GameSession
    exit

  else
    receive whose turn it is
    if it is client's turn then

      loop
        read row and col from user

        if move is valid then
          exit
```

```
        else
          show error message
      send row and col to GameSession

    else
      print that client is waiting for other player to make move

return ""

# startSingleplayerGame
human player starts

loop
  print board
  if there is winner or draw then
    show result
    exit

  if it's player's turn then
    loop
      read row and col from player

      if move is valid then
        exit

      show error message

  else
    show AI thinking dialogue
    AI calculates move

  set piece to board

  flip player turn

return ""
```